Spécification du mode de pilotage automatique

B. Mode automatique (*extrait du dossier de specification du PFR1*)

La partie automatique sera peu détaillée dans ce document, car elle sera approfondie et développée dans le cadre du PFR2.

Fonction : autom()

Entrées : Les données, converties en chaîne de caractères, proviennent de divers capteurs. Ainsi, pour garantir la compatibilité, les paramètres d'entrée sont en format chaîne de caractères.

Sortie : historique des déplacements (fichier enregistrant tous les déplacements).

Actions attendues: déplacement de la plateforme permettant de cartographier la pièce.

Dans ce mode, le robot se déplace de manière autonome en utilisant ses capteurs pour détecter son environnement et éviter les obstacles. Il peut reconnaître des objets spécifiques, comme des balles de couleur, grâce à une caméra embarquée. Le robot génère une carte de son environnement au fur et à mesure de ses déplacements, permettant une visualisation en temps réel de l'espace parcouru. Il est capable de naviguer aussi bien dans des environnements fermés que modifiables, et peut reproduire un trajet précédent si nécessaire.

Schématiquement, nous pouvons représenter ce mode par :

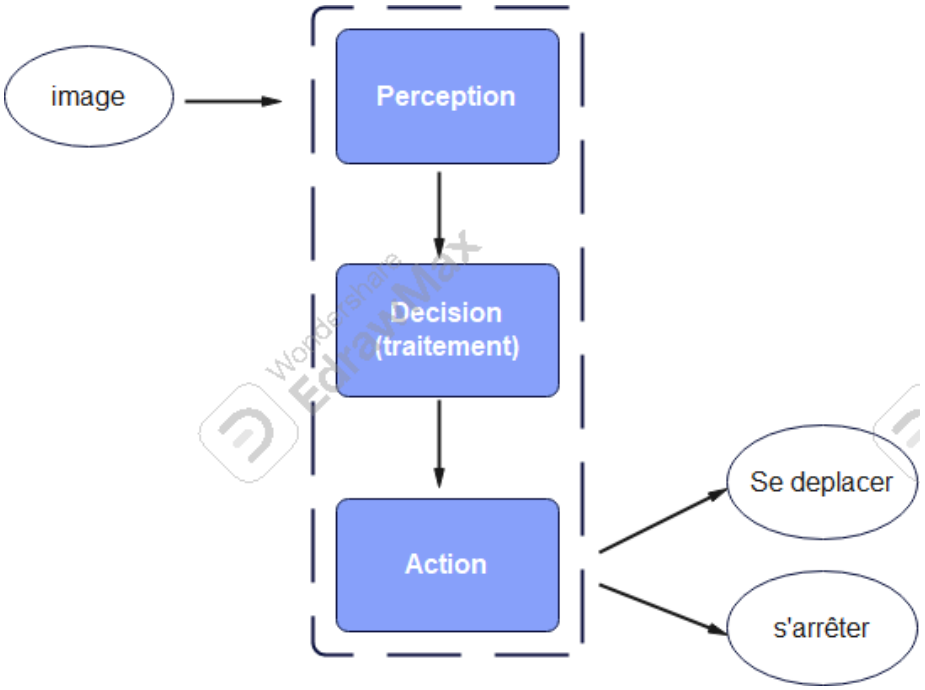


Figure 10 : Représentation du pilotage automatique

En mode automatique, le système doit fonctionner autour du triptyque perception-décision-action. Il perçoit son environnement (images), il traite les données collectées (images, historiques des précédents déplacements, cartographie réalisée précédemment…) puis commande une action qui peut être de se déplacer ou de s'arrêter par exemple.

Ci-dessous, le robot est matérialisé par le camion.

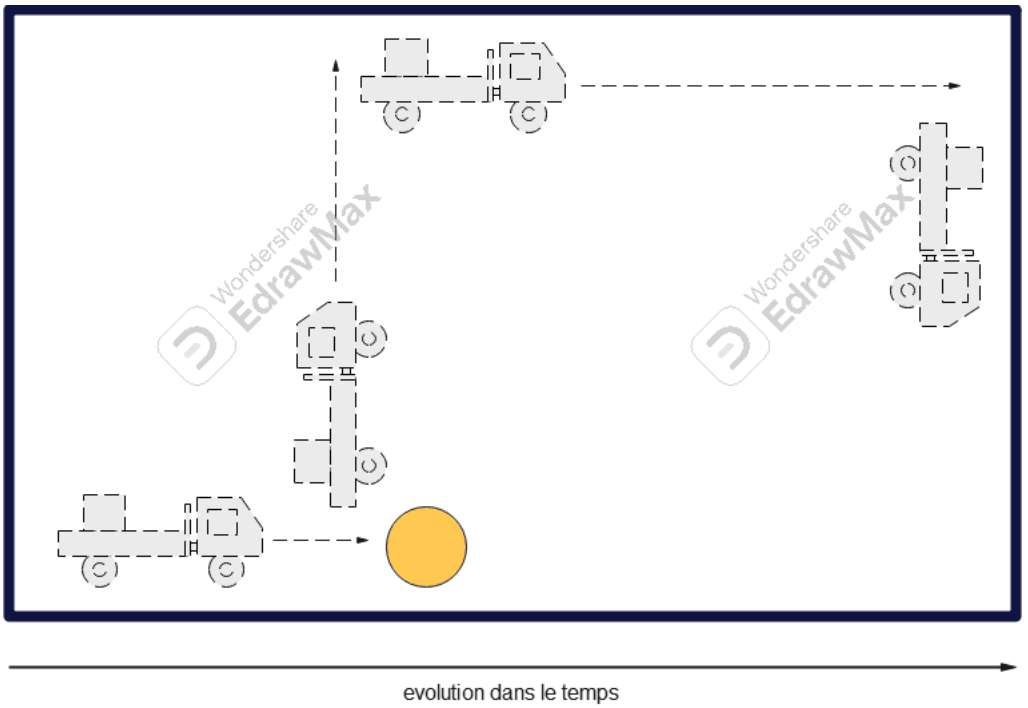


Figure 11 : Scénario pilotage automatique

Exemple de scénario :

- Le robot avance et un capteur (bloc “Perception”) prend une image de ce qu’il y a devant lui et l’envoie en module traitement.
- Le bloc “Traitement” traite l’image et se rend compte qu’il y a une balle devant lui et qu’il ne peut tourner à droite. Après traitement, le module envoie l’action (tourner à gauche) à faire au module suivant.
- Le bloc “Action” reçoit l’information et s’assure qu’elle s’exécute.
- Puis le procédé est répété.

Il est à noter que lorsque plusieurs possibilités de déplacement sont valides, le robot peut en choisir une en fonction de la tâche à effectuer. Par exemple, à la fin du deuxième déplacement de notre robot, il est face à un mur. Mais vu que l’objectif est de cartographier toute la pièce, il décide de tourner à droite pour explorer un plus large espace.

2.1.2. Mode automatique (*extrait du cahier des charges*)

Plus évolué, ce mode suppose que la plateforme dispose de capteurs et d’actionneurs

permettant :

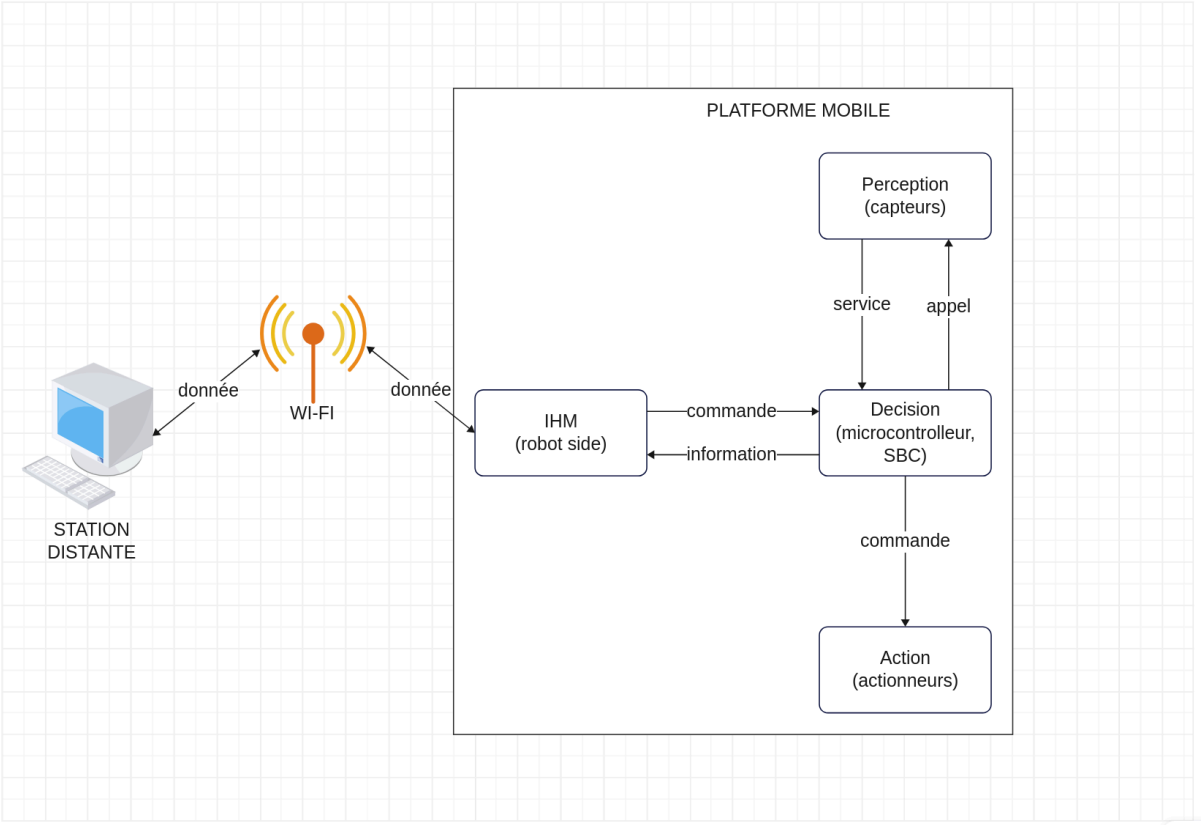
- la localisation dans l’environnement,
- le déplacement,
- la détection d’obstacles (passage impossible) et d’objets spécifiques (balles et objet de couleur présents dans l’environnement et éventuellement recherchés par l’utilisateur),
- et le contournement d’obstacles.

La localisation et le déplacement du robot sont représentés à l’écran sur une carte construite au fur et à mesure (phase de cartographie). Ceci permet une visualisation du périmètre dans lequel évolue le robot (espace libre et obstacles détectés) ainsi que du déplacement réalisé (superposé sur la carte).

Le robot peut identifier un ensemble d’objets par leur forme et leur couleur. Pour cela il est nécessaire d’analyser les images captées par une caméra embarquée sur la plateforme mobile. Cette fonctionnalité est développée dans le cadre du PFR1 et utilisée dans le PFR2.

Enfin, le robot peut évoluer, a minima, dans un environnement fermé, connu à l’avance. Il doit surtout pouvoir évoluer dans un environnement ouvert (modifiable, différent d’une session à l’autre…). Une gestion de l’historique des déplacements est faite pour un éventuel « rejeu » avec par exemple des commandes du type refait le même déplacement.

Architecture globale



- **Bloc Perception :** il rassemble **les yeux et oreilles** de notre plateforme, il collecte les informations sur l'environnement de la plateforme :
  - collecte des données capteurs (données brute),
  - traitement des données brutes fournies par les capteurs (voir tableau de fonctionnalitées).

  Remarque : ce n'est pas au bloc Decision de traiter les données brutes provenant des capteurs. Le bloc Decision ne fait qu'interpréter les données fournies par le bloc Perception.

- **Bloc IHM (robot side) :** c'est le **centre de communication**. Toutes les informations, entre la plateforme mobile et la station distante, transitent par ce module :
  - réception des instructions en provenance de la station distante,
  - envoi des informations que le robot souhaite transmettre à la station distante.
- **Bloc Decision :** C'est le "**cerveau"** du robot, il décide de tout ce que le robot doit faire :
  - actions que le robot doit réaliser en fonction des informations fournies par le module IHM et par les capteurs,
  - notifications à l'utilisateur, mais les informations envoyées passent par le module IHM.

  Remarque : il interprète donc les résultats de l'analyse des capteurs et prend toutes les décisions concernant les actionneurs. Pour cela le bloc Perception lui fournit des services (données capteurs traité) et il peut faire appel à des fonctions du bloc perception.

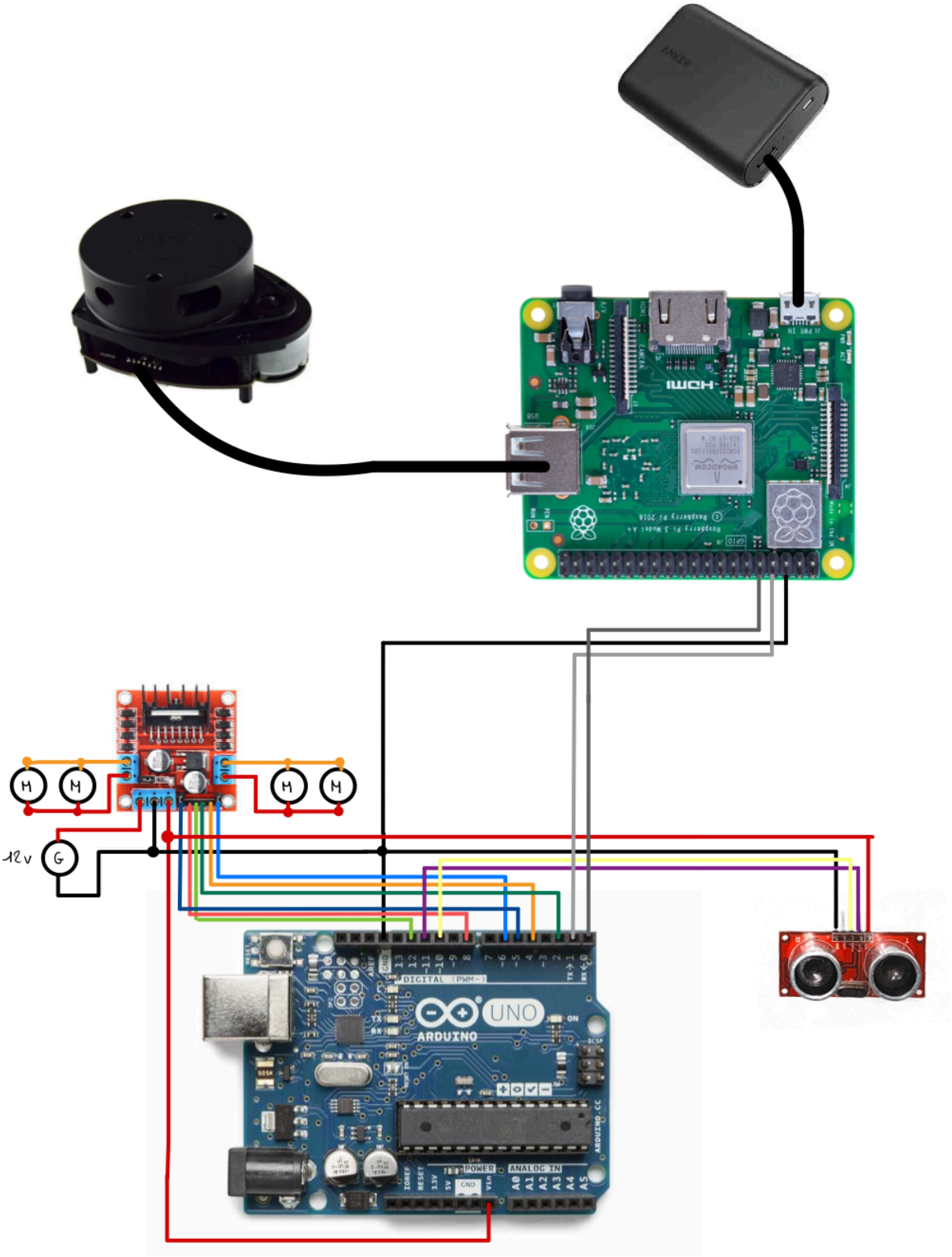
- **Bloc Action :**
    - Il exécute les commandes données par le bloc Decision

  Remarque : aucun contrôle de logique n'est effectué dans ce bloc. Il exécute “**bêtement**” les instructions du bloc action.

Fonctionnalités

Fonctionnalité	Fonction	Nom de la fonction	Entrée	Type d'entrée	Sortie	Type de sortie	Objectifs
Gestion capteurs	caméra	input image()	NONE		image	Image encode (jpg)	Récupérer une image de la scène
		output image()	image encode (jpg)	fichier d'extension jpg	objets correspondant à la description donné	tableau de structure de données.	Reconnaître des formes (balle et cube) de couleurs (bleue, orange, jaune) contenues dans l'image prit en entree
	ultrason	input ultrason()	fréquence à laquelle on reçoit des informations	réels (positif) en ms	données brutes contenant le temps et la distance	données textuelles	Récupérer les données brutes fournies par un ultrason contenant des informations sur la position d'obstacle dans une direction précise.
		output ultrason()	données brutes fournies par un capteur ultrasons	données textuelles	distance de l'obstacle le plus proche dans une direction en cm	Réels	Analyse d'une direction de l'environnement afin de donner la distance a l'obstacle le plus proche du robot a un instant précis.
	Lidar	input lidar()	NONE	NONE	une tuples contenant des informations brut du lidar	tuples: (qualité , angle,distance)	Récupérer les informations fournies par le LIDAR
		output lidar()	données fournies par le capteur lidar	données extraite du flux lidar	Distance et angle de l'objet	vecteur: (angle,distance)	Analyse de l'environnement afin de stocker la position en vue d'un possible appel ultérieur. Mais aussi pour prévenir des obstacles alentour afin d'éviter les collisions (en complément des ultrasons).
Gestion actionneurs	Moteurs	avancer()	distance, vitesse	réel en cm	Le robot avance	une action du robot	Faire avancer (ou tourner) le robot d'une distance (ou d'un angle) spécifique, cette fonctionnalité sera utilisé par les autres fonctionnalité pour les modes manuels et automatique

		tourner()	angle, vitesse de rotation	réel	le robot tourne dans a gauche si l'angle est négatif est à droite sinon	une action du robot	
		reculer()	distance, vitesse	réel en cm	le robot recule d'une certaine distance	une action du robot	
IHM	Commande vocale	IHM_user_to_robot	Commandes vocales	audio	Commande pour le robot	Réel	Traduire les requêtes utilisateurs en commandes compréhensibles par le robot
	Commande manuelle	IHM_user_to_robot	commandes utilisateurs	événementielles (par exemple l'appui ou le click sur un bouton)	Commande pour le robot	Réel	Traduire les requêtes utilisateurs en commandes compréhensibles par le robot
	Feedback	IHM_robot_to_user	Notifications robot		Affichage sur l'IHM user	Texte / graphique	Afficher les informations transmise par le robot
Automatique	cartographie	deplacementAuto()	données fournies par les capteurs	Texte	historique de déplacements, cartographie	Texte	déplacement de robot permettant de cartographier la pièce (de façon automatique)


ANKER
M
M
M
M
12v
G
UNO
ARDUINO
DIGITAL (PWM~)
POWER
ANALOG IN

Algorithme de pilotage autonome

Quels sont les objectifs ?

- Cartographier la pièce : créer une représentation spatiale de son environnement sous forme 2D. L'environnement doit tenir compte de la reconnaissance d'objet.
- Identification d'objet : reconnaitre les balles et cubes bleue, orange et jaune.
- Algorithme

Quels capteurs à disposition ?

Capteurs	**Utilitées**	**Limites**
LIDAR	cartographier la pièce	objets de faible taille
Ultrasons	obstacles dans une direction : avant, gauche,droite	peu de direction prise en compte
Caméra	identification d'objet	objets hors du champ de vision

Présentation de l'algorithme autonome

Le robot embarquera donc

- 01 caméra : elle pointer vers l'avant
- 03 ultrasons
  Il y en aura 01 dans chaque direction : gauche, droite et avant. Ceux de gauche et de droite nous permettront d'évaluer la distance aux obstacles les plus proches du robot.
- 01 LIDAR
  Il sera au-dessus de la plateforme. Il nous permettra d'identifier l'obstacle le plus loin du robot dans une direction donnée

Le fonctionnement de l'algorithme est le suivant : a chaque instant

- **Tâche 1 :** La caméra prend une image de l'avant de la plateforme. L'image prise est ensuite traitée pour en ressortir les objets qui s'y trouvent.
  - Si l'image contient des objets, le robot tourne sur lui-même pour que l'ultrason avant puisse avoir l'objet dans son champ(objet par objet si il y en a plusieurs). Puis les positions de ceux- ci sont renvoyées vers le LIDAR. l'objectif étant de les intégrer à la cartographie. De plus si l'objet detecté est une balle ou un cube de couleur rechrché alors on enregistre la position de celui ci, sa forme et sa couleur.
  - Si l'image ne contient pas d'objet alors il n'y a rien a faire
- **Tâche 2 :** Les ultrasons sur le côté évaluent constamment la distance entre le robot et l'objet le plus proche de celui-ci (sur les côtés gauche et droite). Puis on compare cette distance à celle fourni par le LIDAR.

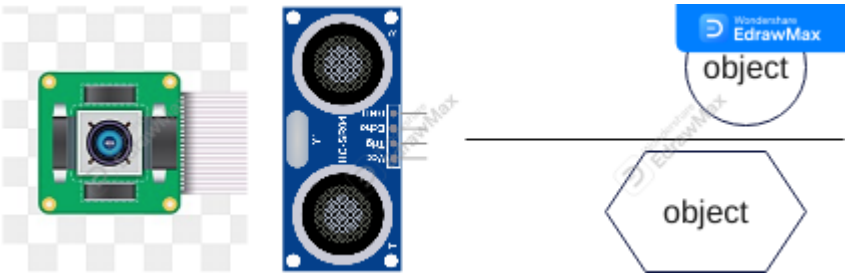
    - Si les distances sont différentes, alors on remonte la position de l'objet en question vers le LIDAR pour l'intégrer à la cartographie. De plus, on reviendra à cette position mais cette fois si en faisant balayer la position en question par l'angle de vue de la caméra …
    - Si les distances sont pareils alors, on ne fait rien
- **Tâche 3 :** Le LIDAR tourne en continu pour mettre à jour les limites de la pièce(objets les plus *éloignés* dans toutes les directions).

NB : ci dessous l'algorithme a été expliqué en évoquant des capteurs en tant qu'organe de traitement. Il est à noter que ce n'est qu'une métaphore. L'objectif étant de montrer la chaîne de traitement des données. De plus, bien qu'il soit précisé tâche 1, tâche 2, tâche 3, les algorithmes associés à ces tâche pourront tourner en parallèles.

Conception détaillée de l'algorithme de pilotage autonome

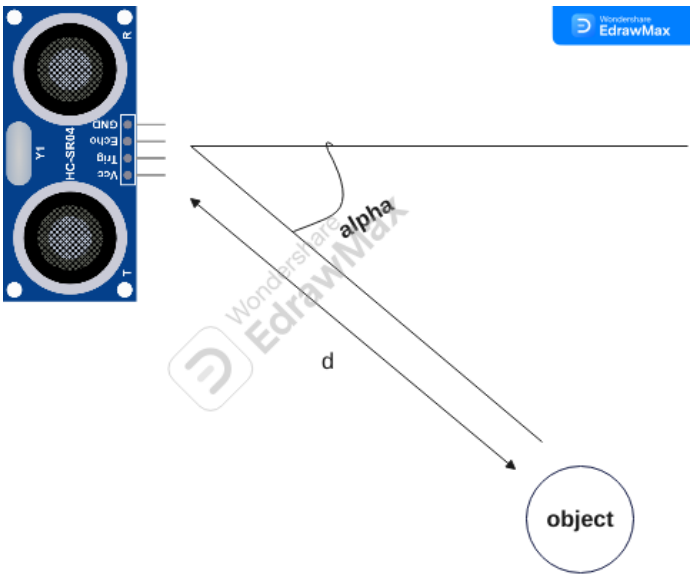
Quels algorithmes à disposition ?

- **Estimation de l'angle de rotation :** nous avons à notre disposition un algorithme utilisant le LIDAR pour estimer de combien de degrés tourne le robot.
- **Estimation de la position d'un objet :** à partir du traitement d'image nous pouvons estimer si un objet se trouve à droite ou à gauche de la direction de l'ultrason avant. Dans l'exemple ci dessous nous serons capable grâce au traitement d'image noter qu'il y a un objet à gauche de la direction de l'ultrason et un second à droite.



axe principal de l'ultrason

- **Matrice de transformation :** nous pouvons ramener un point se trouvant dans le repère mobile associé au robot dans le repère de cartographie.
- **Position des objets dans le repère mobile** : Grâce à l'angle *''alpha''* et a la distance *''d''* on peut déterminer la position de l'objet dans le repère mobile.



Distance dans le repère mobile

Tâche 1

Cette tâche a pour rôle de faire la cartographie de l'avant de la plateforme. Elle combine la caméra, l'ultrason avant et le LIDAR. Nous allons présenter le processus de façon séquentielle : instruction après instruction.

La caméra prend une image. Cette image est ensuite analysée(traitement d'image).

- cas 1 : De(s) objet(s) détectés dans l'image.
  - Si objet(s) a droite de la direction de l'ultrason avant : Alors le robot tourne progressivement sur lui-même vers la droite de 33 degrés.
  - Si objet(s) a gauche de la direction de l'ultrason avant : Alors le robot tourne progressivement sur lui-même vers la gauche de 33 degrés.
  - Si objet(s) dans la direction de l'ultrason avant : la plateforme ne change pas de direction.

  Simultanément, pendant qu'il tourne ou ne tourne pas, l'ultrason avant essayera de repérer à quelle distance sont les objets détectés par le traitement d'image. Cette distance sera comparée plus tard à la distance de l'objet repéré par le LIDAR dans la même direction afin de savoir s' il s'agit bien d'un objet ou d'une limite de la pièce (voir figure tâche 2). De plus, si il s'agit bien d'un objet, on prendra de nouveau une image (centré sur l'objet) et on déterminera la nature de celui-ci.
  Enfin, un traitement de synthèse sera fait. A la fin de traitement, on renverra chaque objet détecté sous la forme :
  - position de l'objet dans le repère mobile (plateforme) ou position dans le repère cartographie
  - Nature de l'objet : balle, cube ou non prise en compte
  - Couleur de l'objet : bleu, orange, jaune ou non prise en compte

  Remarque : Nous tournons de 33 degrés car le champ de vision horizontal de notre caméra est de 66 degrés. Mais nous ne tournerons pas exactement, de 33 degrés car nous devons prendre en compte l'erreur "*rotation error*" liée au l'évaluation de

l'angle de rotation de la plateforme. Nous tournerons donc de $(33 - max(rotation\ error))\ degrés$.

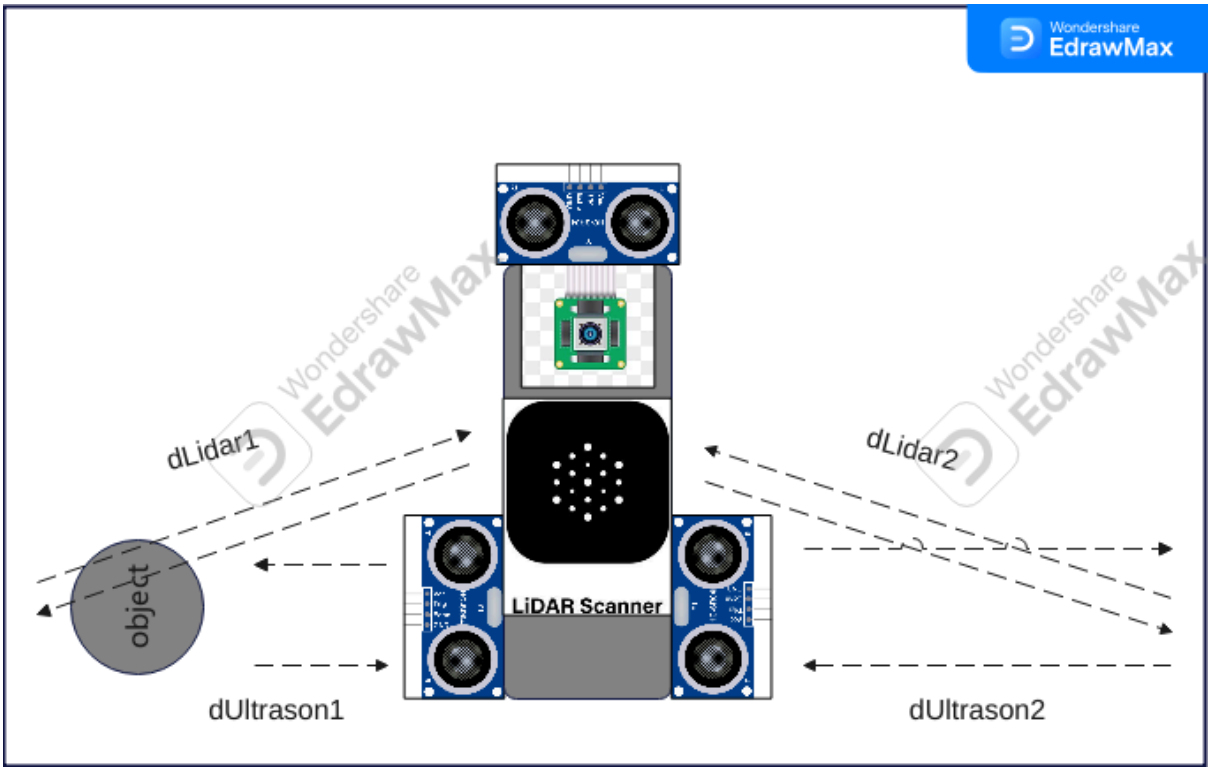
- cas 2 : pas d'objets détectés dans l'image.
  Alors la plateforme continue d'avancer.

Cette tâche est répétée en continue parallèlement au tâches 2 et 3, tant que la cartographie n'est pas terminée.

Tâche 2

Cette tâche peut se diviser en deux parties : comparer puis agir.

- Comparaison :



fusion lidar et ultrason gauche et droit

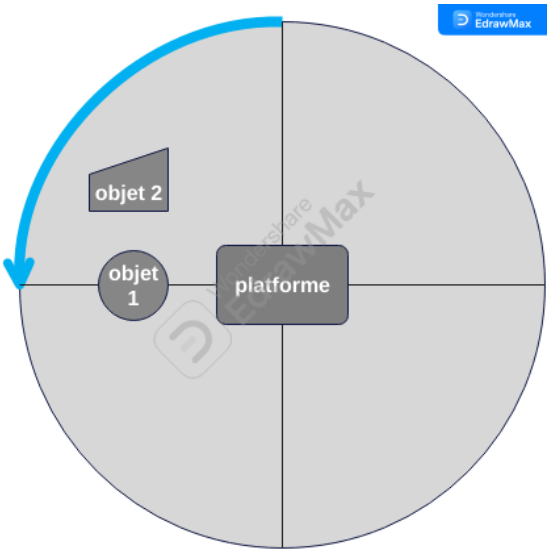
Pour l'ultrason 1 on remarque que $dUlrason\ 1 < dLidar\ 1$ donc il s'agit bien d'un objet et non des limites de la pièce. Dans le cas de l'ultrason 2 on a $dUlrason\ 2 = dLidar\ 2$ donc il s'agit d'une limite de la pièce. Les formules sont donc les suivantes :

- $dUltrason < dLidar \Rightarrow objet$
- $dUltrason = dLidar \Rightarrow limite\ de\ la\ pièce$

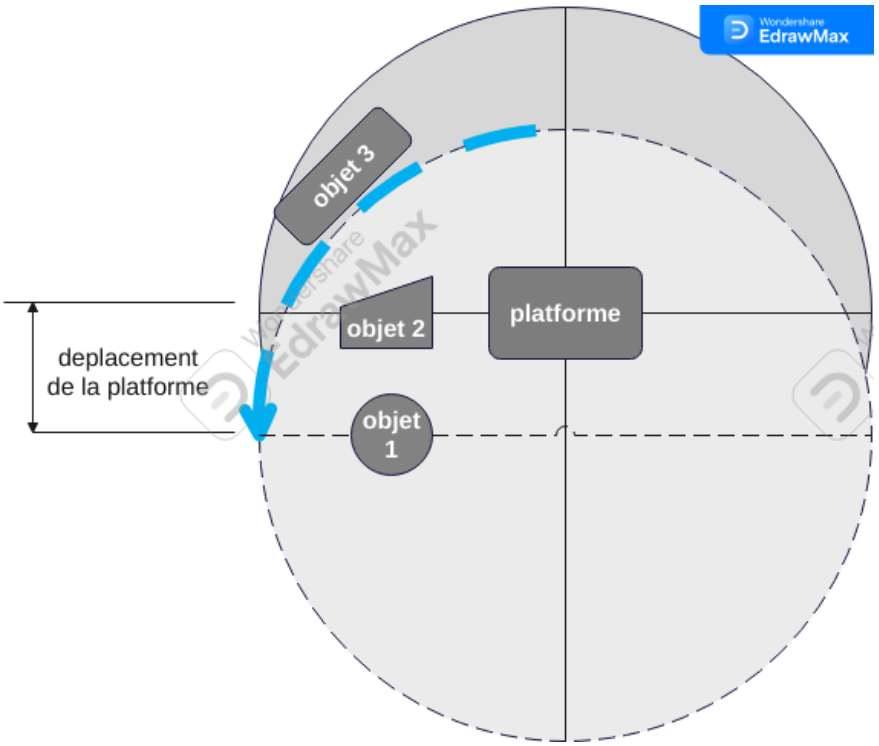
Remarque : Il faudra tenir compte des erreurs. Les erreurs sont liées à la précision de l'ultrason et du lidar.

- Action :
  - Si objet alors : La plateforme tourne de manière progressive d'environ 90 degrés. dans le sens de l'ultrason ayant détecté l'objet. simultanément pendant sa rotation, on combine la caméra, l'ultrason avant et le lidar comme à la tâche 1 pour déterminer la position, nature et couleur des objets rencontrés pendant sa rotation. Après sa rotation (les différents scans

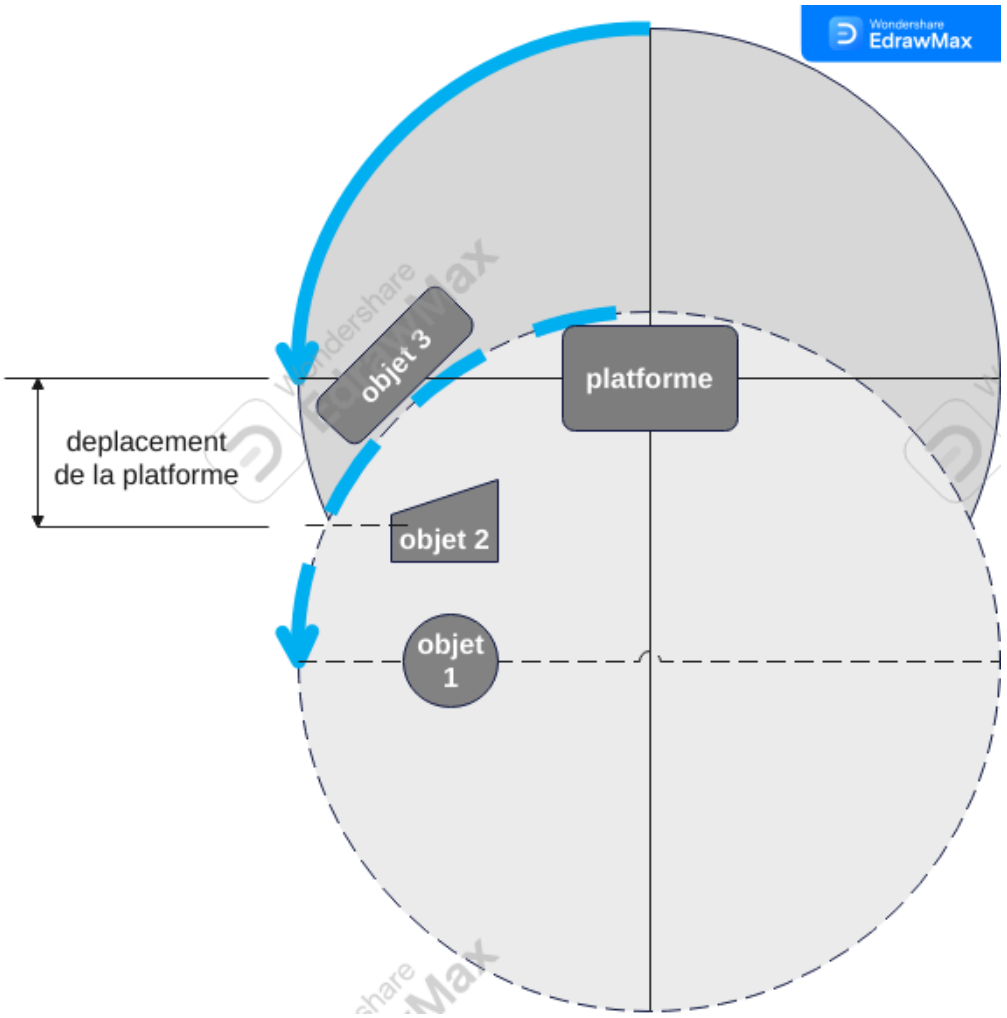
effectués) le robot fera la rotation inverse pour revenir dans la direction d'avant sa rotation.
La difficulté dans cette étape est qu'après avoir terminé son scan (relatif à la rotation d'environ 90 degrés) que le robot ne refasse pas un nouveau scan sur la même zone. Nous illustrons nos propos au travers des schémas ci-dessous.



L'ultrason gauche de la plateforme détecte "*objet 1*". La phase de **comparaison** de la tâche 2 est lancée et il en résulte que "*objet 1*" est bien un objet et non une limite de la pièce. Alors La phase **action** de la tâche 2 est lancée. Le robot tourne comme indiqué par la flèche bleue.
Mais il repère tout d'abord objet 2. La tâche 1 est donc lancée et il enregistre donc "*objet 2*" (position, nature et couleur) à la cartographie. Puis il repère "*objet 1*" et fait de même. Puis fait une rotation inverse pour revenir dans la direction initiale (avant la rotation).



Puis la plateforme recommence à avancer (voir figure ci dessus). L'ultrason gauche de la plateforme détecte "*objet 2*". La phase de **comparaison** de la tâche 2 est lancée et il en résulte que "*objet 2*" est bien un objet.
Mais on se rend compte que "*objet 2*" appartient à une zone ayant déjà été balayée par la plateforme. Alors dans ce cas il n'est pas nécessaire que la plateforme se tourne car "objet 2" a déjà été recensé. Elle continue donc à avancer.



L'ultrason gauche de la plateforme détecte "*objet 3*". La phase de **comparaison** de la tâche 2 est lancée et il en résulte que "*objet 3*" est bien un objet et non une limite de la pièce. Grâce aux informations fournies par l'ultrason gauche on vérifie si "*objet 3*" appartient à une zone ayant déjà été balayée par la plateforme. La conclusion est non. Alors la phase **action** de la tâche 2 est lancée. Le robot tourne comme indiqué par la flèche bleue (celle qui n'est pas en interrompre). Puis repère "objet 3" en ressort sa position, nature et couleur. Ensuite effecte la rotation inverse et continue son chemin.

- Si ce n'est objet alors : la plateforme continue son chemin

Remarque : la tâche 2 tourne tant que la plateforme **avance** (attention, la tâche 2 est désactivée lorsque la plateforme tourne sur elle-même).

Tâche 3

I. Mode Automatique

1. Initialisation

- L'IHM affiche : "Devant l'ouverture, prêt à démarrer"

- Le robot active ces capteurs à ultrasons, LiDAR et caméra. L'IHM indique également l'état des capteurs et de la batterie.

- L'utilisateur peut commencer le déplacement en cliquant sur "Démarrer".

- Le robot commence à se déplacer en mode autonome.

2. Début du déplacement et rencontre d'un balle orange

- Le robot **avance** de 50 cm en ligne droite
- Le **capteur à ultrasons avant** détecte un obstacle à **50 cm**:
    - **IHM** : *"Obstacle détecté à 50 cm."*
    - **Action** : l'obstacle est une balle rouge, la **caméra** confirme que c'est bien une balle orange , et le robot **tourne à droite** pour l'éviter.
    - **IHM** : *"Balle rouge détectée. Tourner à droite."*

- Le robot avance encore de **100 cm**.

3. Rencontre avec un mur

- Après **150 cm** de déplacement, le **LiDAR** détecte un **mur** à **70 cm**, et le **capteur à ultrasons avant** confirme cette détection avec une lecture de distance de **70 cm**.
    - **IHM** : *"Mur détecté à 70 cm."*

- **Action** : Le robot **fait marche arrière** de **20 cm**, puis **tourne à gauche** pour contourner le mur.
    - **IHM** : *"Mur détecté, marche arrière et virage à gauche."*

- Le robot avance ensuite de **80 cm**.

4. Détection d'une balle bleue

1. Le robot continue de se déplacer et, à **200 cm**, le **capteur à ultrasons avant** détecte une **balle bleue** à une distance de **50 cm**.
    - ❖ **IHM** : *"Balle bleue détectée à 50 cm."*

2. **Action** : La **caméra** confirme qu'il s'agit bien d'une balle bleue, et le robot **tourne à gauche** pour l'éviter, puis avance de **50 cm**.
    IHM : *"Balle bleue détectée, trajectoire ajustée*

5. Rencontre avec un mur

- Après 55 cm, le robot détecte un mur avec son LiDAR à 60 cm, et le **capteur à ultrasons avant** confirme la présence du mur.
    - ❖ **IHM** : *"Mur détecté à 60 cm."*

Action : Le robot fait **marche arrière** de 15 cm, puis ajuste sa trajectoire pour tourner à **gauche**.
IHM : *"Mur détecté, marche arrière et virage à gauche."*

- Il **avance** de **80 cm** avant de trouver un nouveau chemin dégagé**.**
    - ❖ **IHM** : *"Avance de 80 cm après virage à gauche."*

6. Détection d'une balle jaune

- À 350 cm, le robot détecte une balle jaune grâce à sa caméra et son capteur à ultrasons avant.
    - ❖ **IHM** : *"Balle jaune détectée."*

- Action : Le robot tourne à gauche pour l'éviter, puis avance de 60 cm.
    - ❖ **IHM** : *"Virage à gauche pour éviter la balle jaune."*

7**. Fin du parcours - Contournement des obstacles terminé**

- Le robot a désormais contourné tous les obstacles détectés (balle rouge, mur, balle bleue, balle verte, mur et balle jaune).

- **Action** : Le robot s'arrête.
    - ❖ L'IHM affiche : *"Parcours terminé, tous les obstacles ont été contournés."*

A voir : . Recherche de la sortie

- Le robot continue sa progression en évitant les obstacles.

- Il avance de **180 cm** et **trouve l'ouverture de sortie**.

- **L'IHM affiche** : *"Sortie détectée, fin du parcours."*

- Le robot s'arrête.
- L'IHM affiche un récapitulatif des objets rencontrés et des obstacles évités.

- LidarLocaliser est la class principale d'appel lidar :
  - scanner : LidarScanner
  - Instance de la classe LidarScanner pour interagir avec le LIDAR.
  - scan_interval : int, optional
  - Intervalle entre les scans à traiter (par défaut 100).
  - map_filename : str, optional
  - Nom du fichier pour sauvegarder la carte (par défaut 'Carte.png').
  - scan_actual : bool, optional
  - Indique si le scan actuel doit être affiché sur la carte (par défaut False).

- Localize() est une méthode de LidarLocaliser qui fournit à chaque itération :
  - la position actuelle du robot (liste [x, y]),
  - la distance devant (dAvant),
  - la distance derrière (dArriere),
  - à droite (dDroite) et à gauche (dGauche).

- addObjet() est une méthode de LidarLocaliser qui ajoute un objet à la cartographie :
  - récuser : list
  - Liste contenant les coordonnées de l'objet à ajouter.

Dans le cadre de notre projet, nous avons été chargés de mettre en place un système de localisation et de cartographie basé sur un capteur LIDAR embarqué sur un Raspberry Pi. Autrement dit, notre mission consistait à permettre au robot de se “repérer” dans l’espace en temps réel, en analysant son environnement à l’aide de milliers de points mesurés chaque seconde.

Ce n’était pas une tâche anodine : il a d’abord fallu faire en sorte que le LIDAR fonctionne correctement avec le Raspberry. Entre les soucis de reconnaissance du port USB, les vitesses de communication à configurer, et la gestion du moteur intégré, nous avons appris à dialoguer avec un capteur exigeant.

Mais une fois les premiers nuages de points affichés, nous avons vite compris qu’il ne suffisait pas de “voir” pour comprendre. Il fallait donner du sens à ces données : où sommes-nous ? Avons-nous bougé ? Quels objets sont fixes ? Quels éléments représentent les murs, les meubles, ou d’autres obstacles ?

C’est là que la partie vraiment intéressante a commencé.

Pour analyser les scans LIDAR, nous avons construit une suite d’outils réunis dans une classe `TransformationUtils`. Elle nous a permis de :

- convertir les **coordonnées polaires en cartésiennes**, et inversement,
- **filtrer les points** pour réduire le bruit,
- **regrouper les données** en clusters cohérents,
- et surtout, calculer la **transformation géométrique** entre deux scans successifs, en utilisant l’algorithme **ICP**(Iterative Closest Point).

L’objectif était simple en théorie : à chaque nouveau scan, estimer si le robot s’est déplacé ou non, et ajuster sa position en conséquence. Mais en pratique, cela demandait une mise en œuvre rigoureuse. Chaque petite erreur dans l’alignement pouvait fausser la carte entière. C’est pourquoi nous avons mis en place un **contrôle de cohérence** basé sur le scan initial, pour valider ou ignorer certaines mesures.

Nous avons ensuite créé une structure appelée `LidarLocaliser`, capable de gérer :

- la récupération des données,
- le traitement via ICP,
- la **construction progressive de la carte**,
- et le suivi de la **position et orientation du robot**.

Et comme une image vaut mille points (ou plus !), nous avons intégré **matplotlib** pour afficher la carte au fur et à mesure, avec la position du robot en vert, les objets ajoutés en couleur, et les obstacles environnants. Chaque mise à jour sauvegarde automatiquement une image de la carte.

🛠️ Les difficultés rencontrées

Le projet ne s'est pas déroulé sans embûches. Parmi les défis les plus marquants :

- Le **démarrage matériel** : faire reconnaître le LIDAR, configurer le port série, et éviter les blocages de communication.
- La **gestion du bruit et des incohérences** dans les données, qui peut fausser la localisation si elle n'est pas traitée avec soin.
- Le **temps réel** : maintenir une boucle d'acquisition fluide tout en traitant des transformations lourdes nécessite une bonne gestion des threads.

✅ Ce que nous avons appris

- Travailler avec un capteur physique en environnement réel, avec tout ce que cela implique de latence, de bruit, d'erreurs possibles.
- Implémenter un **algorithme de localisation robuste** basé sur la géométrie.
- Penser en "système" : du port série jusqu'à l'image finale, chaque étape compte.

Mais surtout, nous avons découvert à quel point voir la carte se construire petit à petit est motivant. On a vraiment l'impression de **donner une perception à la machine**, et de voir notre code "prendre vie".